

Bocconi

COMPUTER SCIENCE

code 30424 a.y. 2024-2025

Lesson 14

Introduction to Control Flow Statements
(decision-making structures)



Università
Bocconi
MILANO

Objectives of the lesson

- Understand what a control structure is
- Get to know main control flow statements in Python
- Learn how to create different types of conditional statements

What is a control structure

- The order in which program statements are executed hangs on a defined logical pattern, which is called **control structure**
- Python has three types of control structures:
 1. **Sequential:** the default mode
 2. **Selection:** used to execute alternative branches within the code
 3. **Repetition:** used to repeat (iterate) a portion of code multiple times



What is a control flow statement

- The control structure of a program is regulated by **control flow statements**
- Python provides various types of control flow statements that can be classified according to the type of control structure that they determine:
 - conditional statements (**if**, **else**, **elif**)
 - loops statements (**while**, **for**, **continue**, **break**)
- Other statements in Python can determine a program's control flow:
 - error handling statements (**try**, **except**)
 - custom function calls

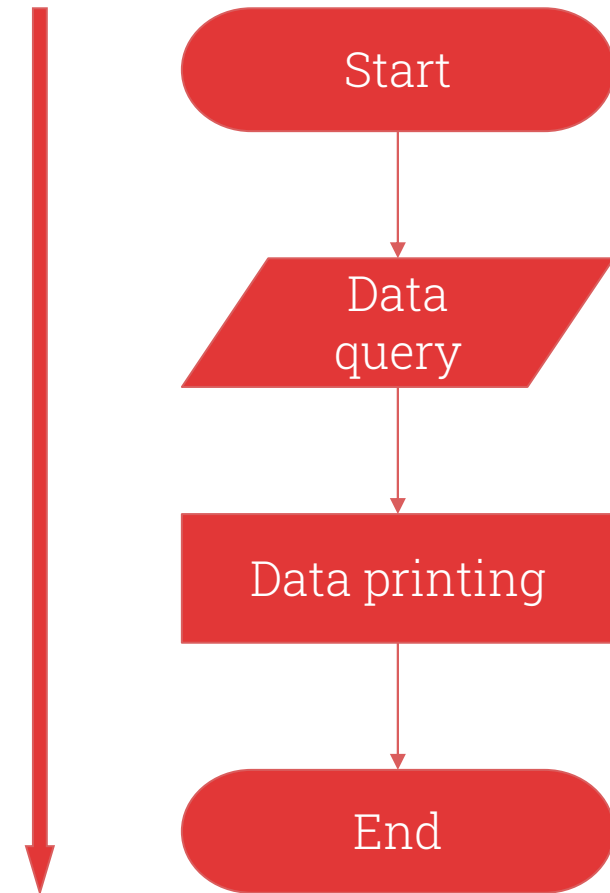


Sequential structures

A sequential structure is characterized by a set of statements which are executed in the order they occur

Example:

```
name = input('What is your name? ')
age = int(input('How old are you? '))
print('Here is the data you entered:')
print('Name:', name)
print('Age:', age)
```



Decision-making structures and conditional statements

In selection structures – also known as **decision-making structures** – a certain portion of code is executed only when specific conditions occur. In Python there are three types of decision-making structures:

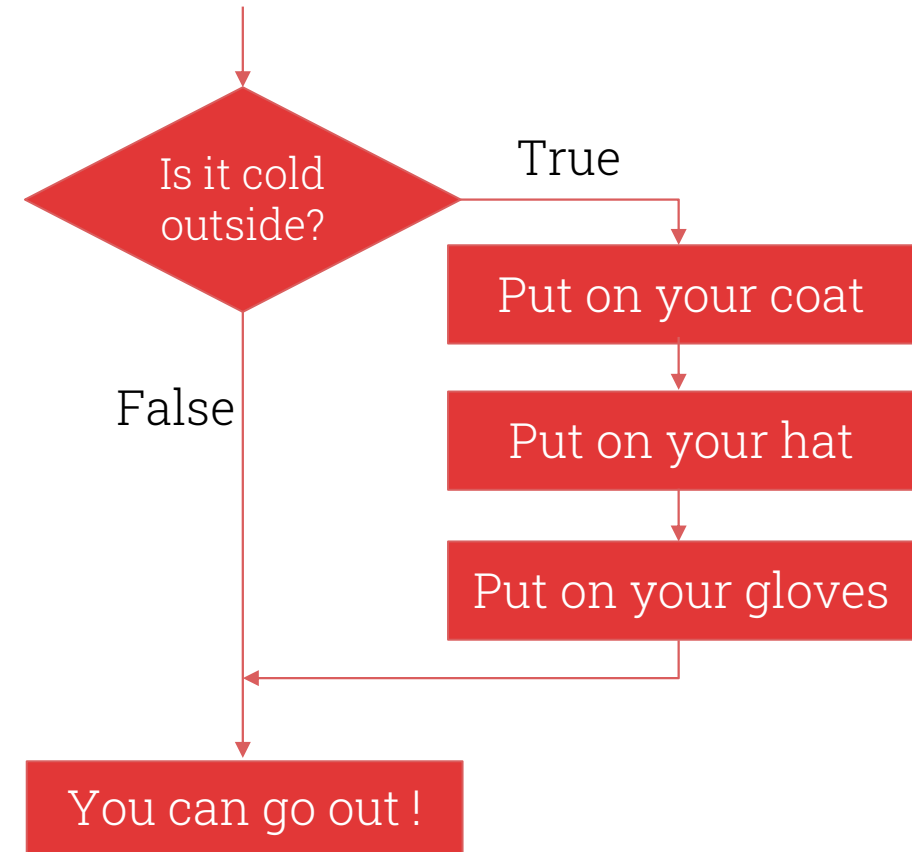
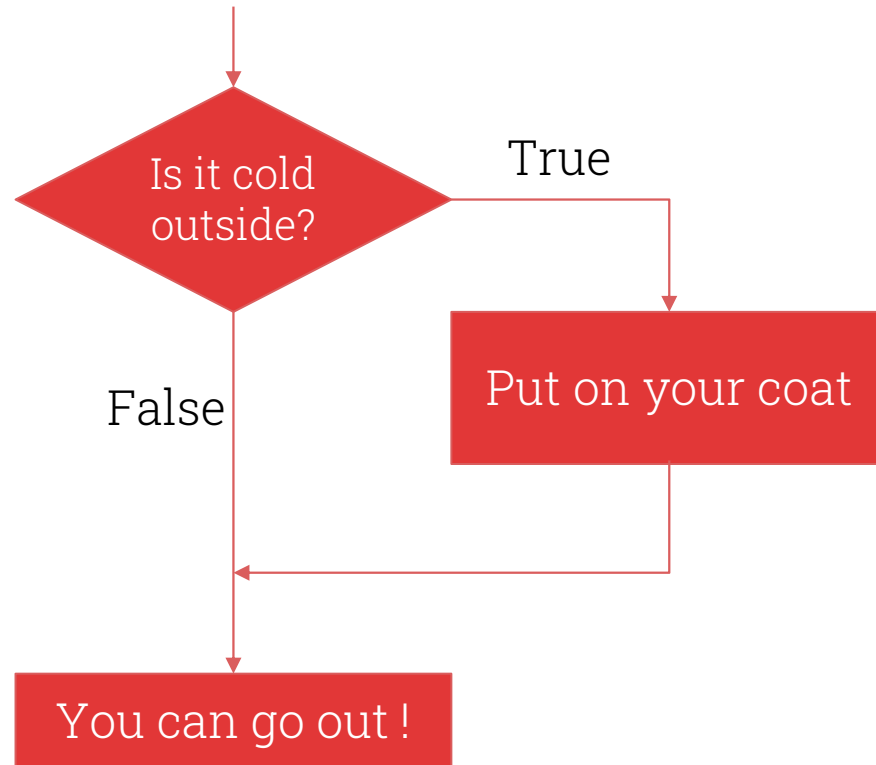
- **Simple** decision-making structure (or single alternative)
- **Alternative** execution (or double alternative)
- **Chained** conditionals

Decision-making structures are implemented by using **conditional statements**.

Python's conditional statements are **if**, **else**, and **elif**.

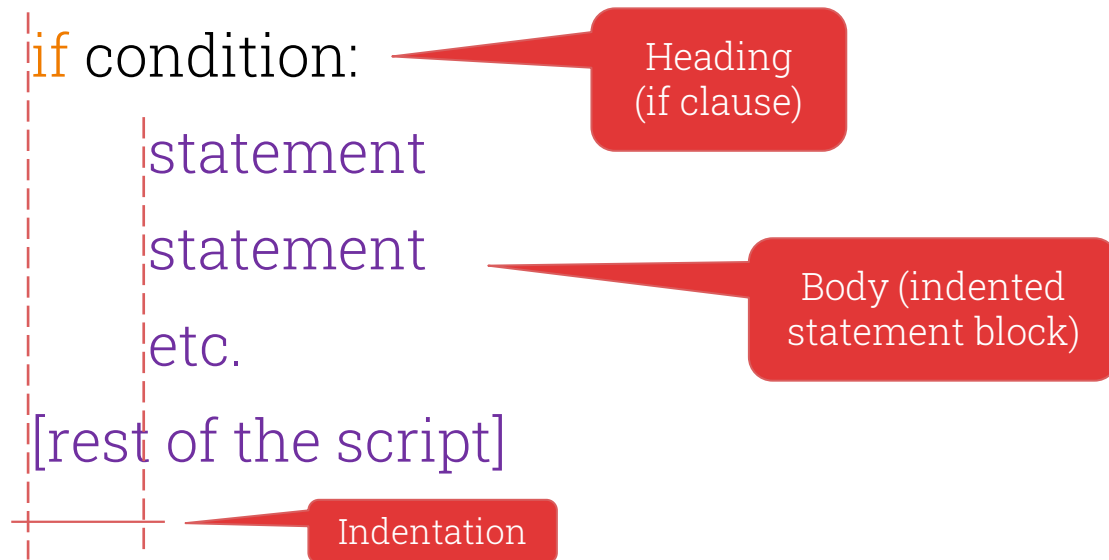
Simple decision-making structure

Examples of a single alternative decision-making structure:



The if statement

In Python, the **if** statement is used to implement a decision-making structure. In the case of a single-alternative structure, the syntax will be:



If the condition is false, the indented block is skipped and the program continues with the rest of the script

Boolean expressions

- The conditions of the **if** statement are implemented by means of **Boolean expressions**, i.e. expressions that can be true or false
- They use comparison operators (<, > etc.) and return **True** or **False**
- **True** and **False** are special values of **bool** type, and they must be typed with a capital letter. They are NOT strings!

```
>>> 5 > 3
True
>>> 3 > 5
False
>>> type(True)
<class 'bool'>
```

Comparison operators

Operator	Description	Examples
<code>==</code>	Equal to	<code>4 == 4 -> True</code> <code>4 == 73 -> False</code>
<code>!=</code>	Different from	<code>4 != 73 -> True</code> <code>4 != 4 -> False</code>
<code>></code>	Greater than	<code>73 > 4 -> True</code> <code>4 > 73 -> False</code>
<code>>=</code>	Greater than or equal to	<code>73 >= 4 -> True</code> <code>4 >= 4 -> True</code>
<code><</code>	Less than	<code>4 < 73 -> True</code> <code>73 < 4 -> False</code>
<code><=</code>	Less than or equal to	<code>4 <= 73 -> True</code> <code>4 <= 4 -> True</code>

The comparison can include values (integers, floats, strings, etc.), variables or functions.

Examples:

```
a >= 5  
length <= width  
city == 'Milan'  
x >= average(y,z)  
...
```



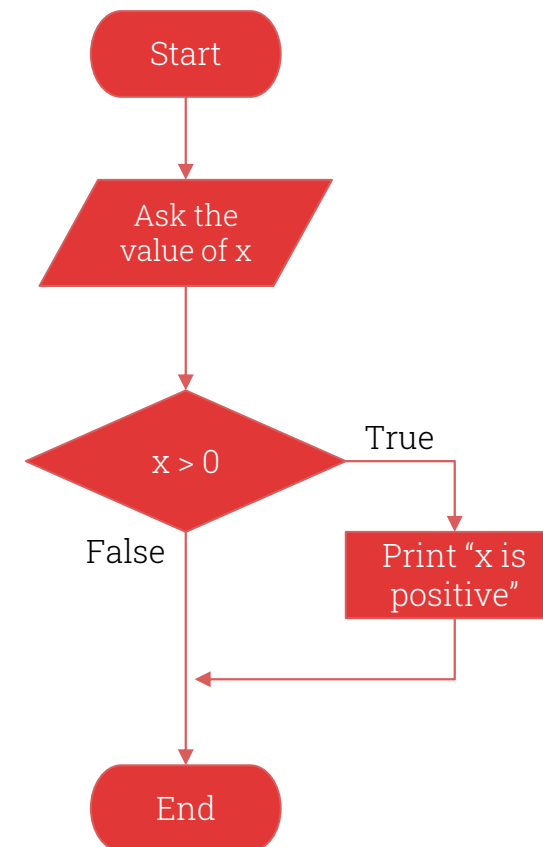
Example of single alternative script

```
x = int(input('Enter the value of x: '))  
if x > 0:  
    print('x is positive')
```

Running the program, it will ask for the x value and it will behave accordingly:

Enter the value of x: 12
x is positive
>>>

Enter the value of x: -3
>>>



Double alternative decision-making structure

- The **if-else** statement defines a **double alternative** decision-making structure, which implies two possible execution paths (**branches**), depending on whether the condition is true or false
- The general syntax for the **if-else** statement is the following:

if condition:

statement
statement
etc.

else:

statement
statement
etc.

[rest of the script]

Example:

Improve the following script to show also 'x is negative or 0'

```
x = int(input('Enter the value of x: '))  
if x > 0:  
    print('x is positive')
```



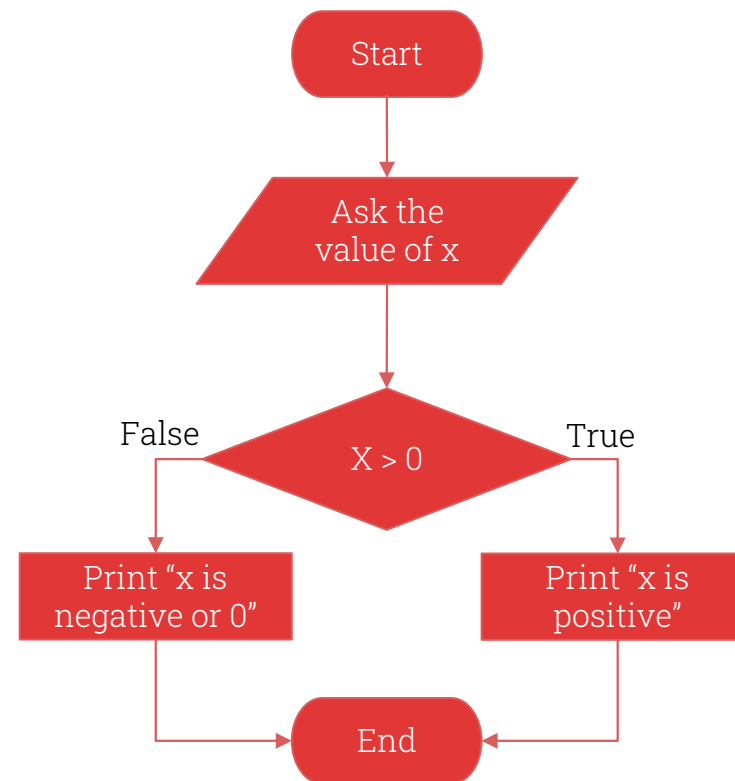
Example of double alternative script

```
x = int(input('Enter the value of x: '))  
if x > 0:  
    print('x is positive')  
else:  
    print('x is negative or 0')
```

Running the program, it will ask for the x value and it will behave accordingly:

Enter the value of x: 12
x is positive
>>>

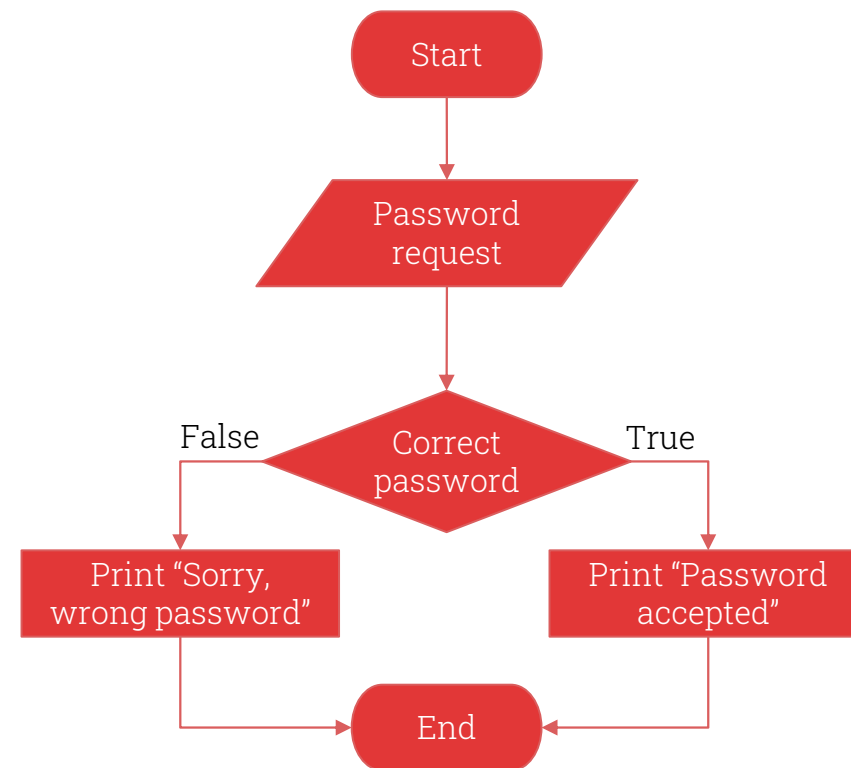
Enter the value of x: -3
x is negative or 0
>>>



Exercise

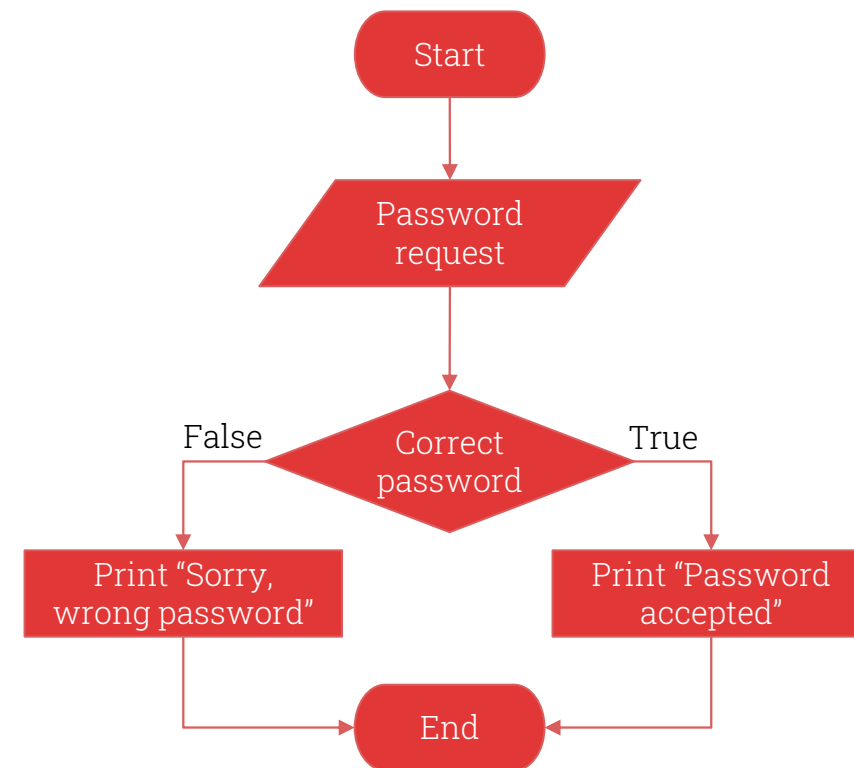
Set up a program which:

- Asks the user to enter a password ("Enter the password:")
- Checks if the password entered by the user is correct, by comparing it to the one saved into the code (i.e. "5ecure")
- Prints "Password accepted", if the password is correct
- Prints "Sorry, wrong password", if the password is not correct



Exercise (solution)

```
password = input('Enter the password: ')\n\nif password == '5ecure':\n    print('Password accepted.')\nelse:\n    print('Sorry, the password is wrong.')
```



Chained conditionals

- When more difficult situations occur (more conditions with more possible outputs), it is possible to nest different **if-else** statements
- In order to verify two conditions with three possible outputs, the syntax is the following:

```
if condition1:
    statements
else:
    if condition2:
        statements
    else:
        statements
[rest of the script]
```

Indentations

Example:

Improve the following script to show also 'x is 0' as a separate output

```
x = int(input('Enter the value of x: '))
if x > 0:
    print('x is positive')
else:
    print('x is negative or 0')
```


Example of chained conditionals script

```
x = int(input('Enter the value of x: '))
if x > 0:
    print('x is positive')
else:
    if x < 0:
        print('x is negative')
    else:
        print('x is 0')
```

Running the program, it will ask for the x value and it will behave accordingly:

Enter the value of x: 12
x is positive

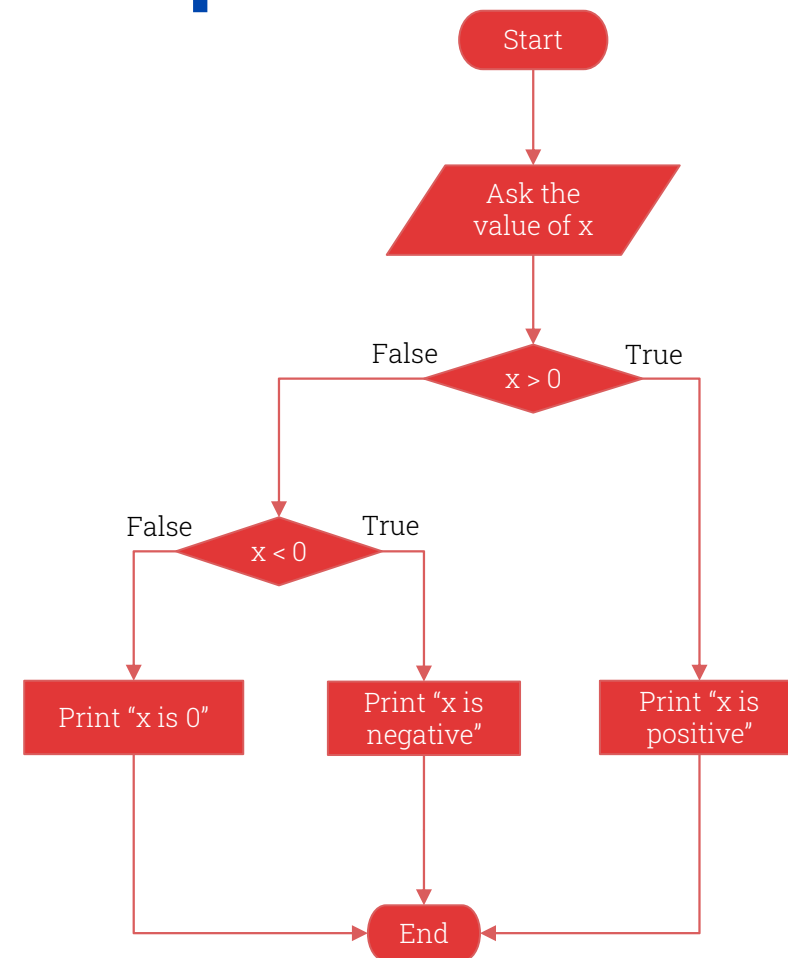
>>>

Enter the value of x: -3
x is negative

>>>

Enter the value of x: 0
x is 0

>>>



Exercise

Set up a program which:

- Requires the user to enter how tall he/she is
("How tall are you (in cm)?")
- Requires the user to enter how tall is the person on his/her right
("How tall is the person on your right (in cm)?")
- Prints one of the following outputs:
 - "You are taller than the person on your right!"
 - "You are shorter than the person on your right!"
 - "What a coincidence: you are tall exactly like the person on your right!"



Exercise (solution)

```
alt1 = int(input('How tall are you (in cm)? '))
alt2 = int(input('How tall is the person on your right (in cm)? '))

if alt1 > alt2:
    print('You are taller than the person on your right!')
else:
    if alt2 > alt1:
        print('You are shorter than the person on your right!')
    else:
        print('What a coincidence: you are tall exactly like the person on your right!')
```

Running the program, it will ask for the two height values and it will behave accordingly:

How tall are you (in cm)? 175

How tall is the person on your right (in cm)? 165

You are taller than the person on your right!

>>>

How tall are you (in cm)? 175

How tall is the person on your right (in cm)? 182

You are shorter than the person on your right!

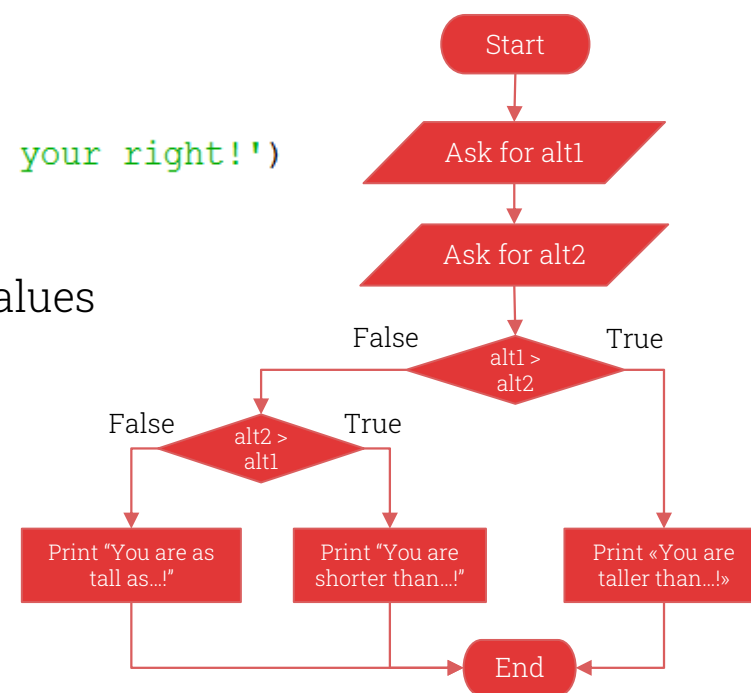
>>>

How tall are you (in cm)? 175

How tall is the person on your right (in cm)? 175

What a coincidence: you are tall exactly like the person on your right!

>>>



The if-elif statement

- In case of **chained conditionals** (when many cases and outputs are possible), the **elif** statement appears to be a more effective alternative to nested **if-else** statements
- **elif** enables to enter a statement for each possible case
- A chain of **elif** can be concluded by an **else** statement to manage residual cases, but it is not compulsory

```
if condition1:  
    statements
```

```
elif condition2:  
    statements
```

```
...
```

```
elif conditionN:  
    statements
```

```
else:  
    statements
```

```
[rest of the script]
```

Exercise

We need to write a program that converts a test score (out of 100) into an American grade (A-F), as shown in the table below:

Score	Grade
91-100	A
81-90	B
71-80	C
61-70	D
0-60	F

The program must print on the screen the final American grade.

Exercise (solution)

We could solve the problem by using nested **if-else** statements, but the code would be long and complex. The **if-elif** structure is more effective:

```
score = int(input('Enter the score: '))

if score >= 91:
    grade = 'A'
elif score >= 81:
    grade = 'B'
elif score >= 71:
    grade = 'C'
elif score >= 61:
    grade = 'D'
else:
    grade = 'F'

print('The grade is', grade)
```

Running the program, it will ask for the score and it will respond accordingly:

Enter the score: 100
The grade is A

Enter the score: 82
The grade is B
>>>



Logical operators

They are also called **Boolean operators** and they enable to create complex Boolean expressions to enhance the potential of conditional constructs:

Operator	Description
and	Returns True when all the arguments are true, otherwise it returns False
or	Returns True when at least one of the arguments is true, otherwise it returns False
not	Returns True if the argument is false and False when it is true



Expressions with logical operators

These expressions always produce **True** or **False** as a result:

Expression	Meaning
$x > y$ and $a < b$	Is x greater than y AND is a less than b at the same time? (True or False)
$x == y$ or $x == z$	Is x equal to y OR is x equal to z ? (True or False)
not ($x > y$)	Is the x > y expression false? (True or False)



Example of if with logical operators

We want to have suggestions before having a walk:

```
rain = input('Is it raining? (Yes or No) ')
temp = int(input('How many degrees are there outside? °C = '))

if rain == "Yes" and temp <=15:
    print('Put on your coat and take the umbrella!')

elif rain == "Yes" and temp >15:
    print('Take the umbrella!')

elif rain == "No" and temp <=15:
    print('Put on your coat!')

else:
    print('Have a nice walk!')
```



The if with in statement

- The logical test of the **if** function can provide a comparison also with a **list of values** or with a **range of numbers**
- The keyword **in** is necessary to use a **list** of values

Example:

```
if rain in ["Yes", "yes", "YES", "y", "Y"] and temp <=15:  
    print('Put on your coat and take the umbrella!')
```

To create a range of values, the **range** function must be used

The range function

- Syntax: `range(start, stop [, step])`
- The `range` function returns an object containing a sequence of integers from **start** (inclusive) to **stop** (exclusive) by increments of **step**

`range(i, j)`

produces: $i, i+1, i+2, \dots, j-1$

- When **start** is omitted, the default value is 0

`range(4)`

produces: 0, 1, 2, 3

- **step**, if specified, determines the increment (or decrement)

Example of if with range

```
a = 12
```

```
if a in range(6,12):
```

```
    print('The value is OK!')
```

```
else:
```

```
    print('The value is not in the range')
```

```
The value is not in the range
```

```
>>>
```

Exercise 1

Create a new Python file named **Exercise 1.py** which contains a program that:

- asks the user to enter two integers (the second one should be greater than the first one)
- shows on screen the sum, the difference and the average of the two numbers

Show a message on the screen if the user enter twice the same number and if the user enters a second number which is smaller than the first one (create a custom message for each single case).

Exercise 2

The file **Exercise 2.py** contains a program to solve a problem related to triangles. The program must:

- ask the user for three numbers (with or without decimals), corresponding to the lengths of the sides of a triangle
- verify that all numbers entered are positive and that, for all possible combinations, each side is less than the sum of the other two (otherwise the lengths provided could not generate a triangle)
- if the previous check is positive, the program prints the message “With these measures, it is ...” followed by the triangle type, i.e. equilateral (when all the sides are equal), isosceles (when two sides are equal) or scalene (when all sides are different)

Complete the program by writing the missing lines of code.

Tools to learn with today's lecture

- Conditional constructs: `if`, `if-else`, `if-elif-else`
- Comparison operators
- Logical operators
- `range` function

Book references

Understanding Python:

Chapter 5

Python workbook:

Exercises of Unit 2